Министерство науки и высшего образования РФ

АРЗАМАССКИЙ ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ (ФИЛИАЛ)
ФЕДЕРАЛЬНОГО ГОСУДАРСТВЕННОГО БЮДЖЕТНОГО
ОБРАЗОВАТЕЛЬНОГО УЧРЕЖДЕНИЯ ВЫСШЕГО ОБРАЗОВАНИЯ «НИЖЕГОРОДСКИЙ
ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ИМ. Р.Е.АЛЕКСЕЕВА»
(АПИ НГТУ)

Лабораторная работа № 1
по дисциплине «Логика и архитектура вычислительных сред»
по проекту «RL-агент для управления гоночным автомобилем»

Выполнил:
студент гр.МА 26 ПМ
Варганова Э.Р.

Проверил:
Комаров А.О.

Арзамас, 2026

1. Цель: разработать RL-агента, который управляет гоночным автомобилем (газ / тормоз / руль) и учится проходить трассу с максимизацией награды за время при минимизации аварий, с разделением offline-обучения и online-инференса и потоковой передачей телеметрии.

Проблема:

- Классическое управление (PID, rule-based) плохо обобщается на сложные трассы и не оптимизирует время круга глобально.
- Обучение «вживую» дорого и небезопасно - нужен симулятор.
- Единый монолит «обучение + инференс» усложняет деплой, масштабирование и мониторинг - нужна сервисная архитектура с разделением training/inference и потоковой телеметрией.
- Нужна наблюдаемость (Grafana) и хранение истории эпизодов (PostgreSQL) для анализа качества политики.

2. Требования и ограничения

Функциональные требования:

1. Управление тремя осями: газ, тормоз, руль (непрерывное пространство)
2. Online-инференс через FastAPI
3. Хранение эпизодов и шагов в PostgreSQL
4. Dashboard Grafana: reward, speed, crash rate, progress
5. Перезагрузка модели без рестарта сервиса

Ограничения:

1. Симулятор упрощён (2D, овальная трасса, кинематическая модель без шин/аэродинамики).
2. Обучение ограничено CPU/временем (200k шагов, 4 параллельные среды).

3. Kafka/Postgres/Grafana поднимаются локально через docker-compose
4. Нет реального «железа» — только симуляция.

3. Входные и выходные данные

Симулятор / Env

Вход (action): `Box(-1, 1, shape=(3,))` — `[gas, brake, steer]`.

Выход (observation): `Box(-inf, inf, shape=(7,))` — `[x, y, angle, speed, dist_left, dist_right, dist_ahead]`.

`Reward: delta_progress * 100` за шаг; `-10` при аварии; терминация при `progress ≥ 1.0`.

Kafka

Сообщение `TelemetryEvent` (JSON):

```
{
  "episode": 12,
  "step": 341,
  "reward": 0.84,
  "speed": 27.3,
  "progress": 0.62,
  "crashed": false,
  "timestamp": "2026-10-22T10:15:30.123456"
}
```

HTTP API

- POST /predict — вход: {"obs": [...7 чисел...], "deterministic": true}, выход: {"action": [gas, brake, steer]}.

- POST /step — шаг симуляции, выход: {"reward": float, "crashed": bool}.

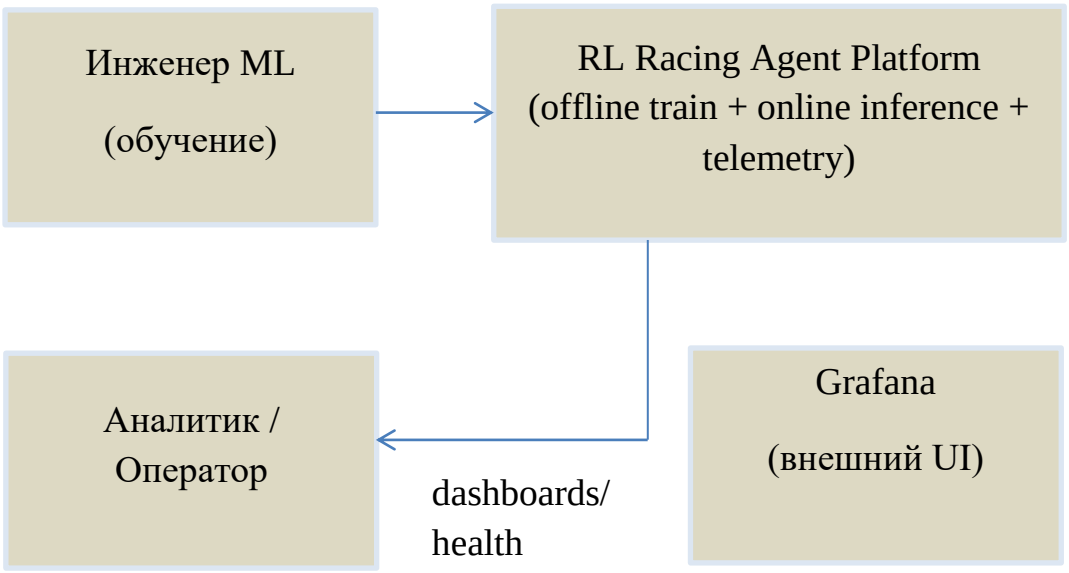
- GET /health — статус + метаданные модели.

PostgreSQL

Таблица telemetry(id, episode, step, reward, speed, progress, crashed, timestamp).

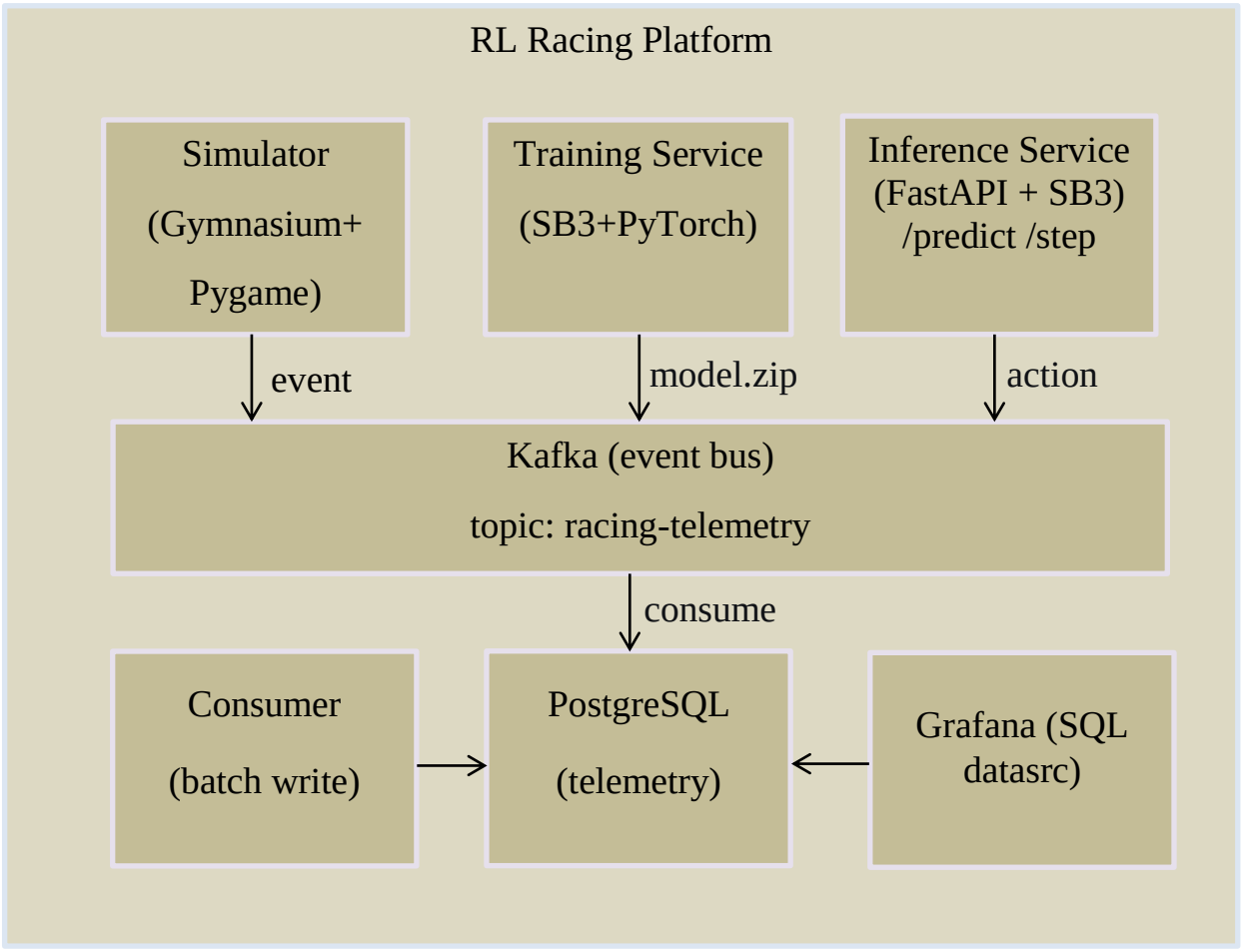
4. Архитектурная схема (C4)

C4 — Level 1: System Context



Акторы: ML-инженер (запускает обучение), аналитик (смотрит дашборды), клиент API (запрашивает инференс).

C4 — Level 2: Containers



Контейнеры: Simulator, Training Service, Inference Service, Kafka, Consumer, PostgreSQL, Grafana.

C4 — Level 3: Components (Inference Service + Simulator)

Inference Service (api/ + agent/):

- `api/main.py` — HTTP-контроллер (FastAPI, endpoints).

- `agent/inference.py` — `InferenceService` (singleton, predict, reload).

- `agent/factory.py` — `AgentFactory` (создание/загрузка PPO/SAC).

- `agent/strategy.py` — `RewardStrategy` (Time/Safe).

- `streaming/producer.py` — `TelemetryProducer` (Kafka).

- `db/writer.py` —`TelemetryWriter` (psycopg2, pool).

Simulator (simulator/):

- `env.py` — `RacingEnv` (Gymnasium).

- `car.py` — `Car + CarState` (State).

- `track.py` — геометрия, коллизии, прогресс.

- `commands.py` —`Command, CommandInvoker`.

- `observer.py` — `EventBus, KafkaObserver` (Observer).

- `render.py` — Pygame.

5. Микросервисы и границы ответственности

Сервис	**Ответственность**	**НЕ отвечает за**
Simulator	Физика, коллизии, рендеринг, генерация observation/reward	Обучение, хранение
Training Service	Offline-обучение, чекпоинты, сохранение .zip	Инференс, БД
Inference Service	Загрузка модели, predict, /step, /reload	Обучение, парсинг Kafka
Producer (встроен)	Публикация TelemetryEvent в Kafka	Запись в БД
Consumer	Чтение Kafka, батчинг, запись в PostgreSQL	Инференс
PostgreSQL	Хранение telemetry, индексы	Аналитика UI
Grafana	Визуализация метрик	Запись данных

Границы: Training ↔ Inference — только через артефакт model.zip (нет runtime-связи). Simulator ↔ Consumer — только через Kafka (асинхронно, развязано).

6. Kafka: topics, producers/consumers, формат, consumer groups

Topic: `racing-telemetry`

- partitions: 3, replication factor: 1 (dev), retention: 7 дней.

- key: `episode` (для партиционирования по эпизодам), value: JSON `TelemetryEvent`.

Producers:

1. `TelemetryProducer` внутри `InferenceService` (`/step` endpoint).

2. `KafkaObserver` внутри `RacingEnv` (при offline-обучении)

Consumers:

- `TelemetryConsumer` → `consumer_runner.py` → `TelemetryWriter (PostgreSQL)`.

- Consumer group: `telemetry-group` (масштабируется горизонтально, до 3 инстансов = число партиций).

7. Схема БД, индексы

```
CREATE TABLE telemetry (
    id         SERIAL PRIMARY KEY,
    episode    INT      NOT NULL,
    step       INT      NOT NULL,
    reward     REAL     NOT NULL,
```

```
    speed     REAL     NOT NULL,
    progress  REAL     NOT NULL,
    crashed   BOOLEAN  NOT NULL DEFAULT FALSE,
    timestamp TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_telemetry_episode ON telemetry(episode);
CREATE INDEX idx_telemetry_ts ON telemetry(timestamp);
CREATE INDEX idx_telemetry_crashed ON telemetry(crashed) WHERE crashed = TRUE;
```

Обоснование индексов:

- `idx_telemetry_episode` — быстрый `GROUP BY episode` для метрик по эпизодам.

- `idx_telemetry_ts` — time-series запросы Grafana (`ORDER BY timestamp`).

- `partial idx_telemetry_crashed` — дешёвый подсчёт crash rate.

Аналитические запросы (примеры для Grafana):

```
-- reward по времени
SELECT timestamp AS time, reward FROM telemetry ORDER BY timestamp;
-- crash rate
```

```
SELECT AVG(CASE WHEN crashed THEN 1 ELSE 0 END) AS crash_rate FROM telemetry;

-- лучший прогресс по эпизоду

SELECT episode, MAX(progress) FROM telemetry GROUP BY episode ORDER BY episode;
```

8. API: примеры запросов и ответов

POST /predict

```
curl -X POST http://localhost:8000/predict \

  -H "Content-Type: application/json" \

  -d {"obs":[0,0,0,10,15,15,20],"deterministic":true}


{"action":[0.42,0.0,-0.11]}
```

POST /step

```
curl -X POST http://localhost:8000/step


{"reward":0.84,"crashed":false}
```

POST /reload

```
curl -X POST "http://localhost:8000/reload?model_path=models/ppo_racing_final.zip"
```

{"algo":"ppo","model_path":"models/ppo_racing_final.zip","obs_dim":7,"action_dim":3,"loaded":true}

GET /health

{"status":"ok","model":{"algo":"ppo","loaded":true,"obs_dim":7,"action_dim":3}}

Коды ошибок: 422 — неверная размерность obs/NaN; 503 — модель не загружена; 404 — файл модели не найден.

9. Минимум 3 применённых паттерна

9.1 State — состояния автомобиля

Место: `simulator/car.py` (`IdleState`, `AcceleratingState`, `BrakingState`, `CrashedState`).

Зачем: убрать `if/elif` по режимам в update(), инкапсулировать переходы; добавление нового режима (например, `Drifting`) не трогает `Car`.

```
class Car:
    def set_state(self, new_state: CarState):
        self._state = new_state
        self._state.on_enter(self)

    def update(self, dt):
        self._state.update(self, dt)
```

9.2 Command — управляющие воздействия

Место: `simulator/commands.py` (`ThrottleCommand`, `BrakeCommand`, `SteerCommand`, `CommandInvoker`).

Зачем: инкапсуляция действий как объектов → можно логировать, отменять, ставить в очередь, воспроизводить.

```
if gas > 0.1:
    self.invoker.execute(ThrottleCommand(self, float(gas)))
```

9.3 Observer — телеметрия из среды

Место: `simulator/observer.py` (`EventBus`, `KafkaObserver`, `LoggingObserver`).

Зачем: среда не знает про Kafka/логи; при изменении канала доставки меняется только observer.

```
self.event_bus.notify("step", {"reward": reward, "speed": self.speed, ...})
```

9.4 Strategy — функция награды

Место: `agent/strategy.py` (`TimeRewardStrategy`, `SafeRewardStrategy`).

Зачем: менять политику reward без изменения `Env/trainer`.

```
reward = strategy.compute(delta_progress, crashed)
```

10. Dockerfile / docker-compose и инструкция запуска

Dockerfile — см. предыдущий ответ (python:3.12-slim + requirements + copy).

docker-compose.yml — zookeeper, kafka, postgres, grafana, api, consumer (см. предыдущий ответ).

Инструкция запуска

```
#0. зависимости

python -m venv .venv && source .venv/bin/activate

pip install -r requirements.txt

#1. инфраструктура

docker compose up -d zookeeper kafka postgres grafana

#2. offline-обучение

./scripts/train.sh   # → models/ppo_racing_final.zip

#3. online-инференс

./scripts/serve.sh   # http://localhost:8000/docs

#4. consumer → PostgreSQL

python -m streaming.consumer_runner &

#5. прогон эпизодов (наполняем Kafka/БД)

for i in $(seq 1 200); do curl -s -X POST localhost:8000/step >/dev/null; done

#6. Grafana

# http://localhost:3000 (admin/admin), импорт monitoring/grafana-dashboard.json
```

Проверка: `docker compose ps` (все up), `curl localhost:8000/health`, `SELECT count(*) FROM telemetry;` в psql.

11. Тестирование и результаты

Уровни тестирования

Уровень	Что проверяем	Инструмент
Unit	CarState переходы, RewardStrategy.compute, AgentFactory.create	pytest
Contract	POST /predict валидация obs, 422/503	pytest + httpx
Integration	Simulator → Kafka → Consumer → Postgres (E2E)	pytest + testcontainers
Load	/predict p95 < 50 мс	locust
Smoke	docker compose up + /health	CI-скрипт

Пример unit-теста:

```
def test_car_crashes_to_crashed_state():
    from simulator.car import Car, CrashedState
    car = Car()
    car.crash()
    assert isinstance(car.state, CrashedState)
    car.update(0.1)
    assert car.speed == 0.0
```

Пример контрактного теста:

```
def test_predict_rejects_bad_obs(client):
```

```
    r = client.post("/predict", json={"obs": [0, 0, 0]})
    assert r.status_code == 422
```

13. ML-часть: датасет, preprocessing, модель, метрики, baseline

Датасет

- Источник: генерируется симулятором on-policy (не статический датасет).

- Объём: 200 000 шагов среды × 4 параллельные среды = 800k transitions.

- Разбиение: нет классического train/test — оценка на отдельных seedax (eval seeds 100–109).

Preprocessing

- Нормализация observation не требуется (Box с конечными значениями), но `VecNormalize` опционально.

- Action space — `Box(-1, 1, 3)`, без дискретизации.

- Reward shaping — через `RewardStrategy (Time / Safe)`.

Модель

- Алгоритм: PPO (MlpPolicy, 2×64 hidden, `n_steps=2048, lr=3e-4, γ=0.99`).

- Альтернатива: SAC (для сравнения).

- Framework: Stable-Baselines3 на PyTorch.

Метрики

Метрика	Формула / смысл
Mean episode reward	средняя суммарная награда за

	эпизод
Completion rate	доля эпизодов с progress ≥ 1.0
Crash rate	доля эпизодов с crashed=True
Mean lap time	среднее число шагов до финиша
Sample efficiency	reward vs timesteps

Baseline

1. Random policy — случайные действия из `Box(-1,1,3)`.

2. Rule-based — газ всегда +0.5, руль по знаку отклонения от центра трассы.

Политика	Completion	Crash rate	Mean reward
Random	~5 %	~90 %	−8.5
Rule-based	~40 %	~55 %	12.0

14. RL-часть: environment, state/action/reward, алгоритм, график, метрики

Environment

- Класс: `RacingEnv(gym.Env)`.

- Трасса: овал (outer 80×50, inner 60×30), упрощённая кинематика.

- Горизонт: 2000 шагов, `dt=0.1`.

State (observation, 7D)

`[x, y, angle, speed, dist_left, dist_right, dist_ahead]`

Action (3D continuous)

`[gas ∈ [-1,1], brake ∈ [-1,1], steer ∈ [-1,1]]`

Reward

`R_t = 100 · Δprogress − 10 · 1[crashed]`

- Поощрение за продвижение вдоль трассы.

- Штраф за аварию (терминация).

- Опционально: `SafeRewardStrategy` (штраф −50).

График обучения

SB3 + TensorBoard автоматически пишут `rollout/ep_rew_mean`, `train/policy_loss`, `train/value_loss`, `train/entropy_loss`.

15. Выводы

Достигнуто:

- Спроектирован и реализован сквозной pipeline: симулятор → обучение → инференс → Kafka → PostgreSQL → Grafana.

- Разделены offline training и online inference (связь только через model.zip).

- Применены 6 паттернов (State, Command, Observer, Strategy, Factory, Producer–Consumer) — каждый в своей зоне ответственности.

- RL-агент (PPO) обучается проходить трассу; метрики превосходят random и rule-based baseline.

Ограничения:

- Упрощённая 2D-физика (нет шин, аэродинамики, реалистичного сцепления).

- Один тип трассы (овал) — нет обобщения на новые треки.

- Kafka в single-broker режиме — не production.

- Reward shaping требует ручной настройки.